

运算符

算术运算符

关系运算符

逻辑运算符

赋值运算符

*位运算符

其他：条件运算符 逗号运算符

算术运算符

+ - * / %

整数支持 + - * / %

浮点数不支持取余数%

```
#include <stdio.h>
int main() {
    //int a = 10, b = 3;
    //printf("a + b = %d\n", a + b);
    //printf("a - b = %d\n", a - b);
    //printf("a * b = %d\n", a * b);
    //printf("a / b = %d\n", a / b);
    //printf("a %% b = %d\n", a % b); //printf显示%得写%%

    //float a = 10, b = 3;
    //printf("a + b = %f\n", a + b);
    //printf("a - b = %f\n", a - b);
    //printf("a * b = %f\n", a * b);
    //printf("a / b = %f\n", a / b);
    //printf("a %% b = %f\n", a % b); 取余运算符操作数必须是整数

    int a = 1, b = 2, c = 3;
    // 优先级 * / % 高于 + -
    // 相同优先级按照从左往右的顺序 结合性
    printf("output = %d\n", a + b * c);
    return 0;
}
```

关系运算符

C语言当中如何描述真和假

数据机器数是0 ---> 假的

数据机器数是非0 ---> 真的

} → if 结构.

关系运算符

判断相等关系 判断相等== 判断不相等!=

判断大小关系 < <= > >=

满足条件，返回值就是1；不满足条件，返回值就是0

```
int a = 1, b = 2, c = 1;
printf("a == b < c is %d\n", a == b < c); // 先做< 再==
printf("(a == b) < c is %d\n", (a == b) < c);
return 0;
```

Microsoft Visual Studio 调试控制台

```
a == b < c is 0
(a == b) < c is 1
```

判断大小的运算符优先级要高于判断相等性，结合性是从左往右

逻辑运算符

对布尔表达式（返回真或者假的表达式）做运算

逻辑与&& 逻辑或|| 逻辑非!

逻辑与 双目 L&&R

L	R	N
1	1	1
1	0	0
0	1	0
0	0	0

逻辑或 双目 L||R 逻辑非 单目 !0-->1 !1-->0

L	R	N
1	1	1
1	0	1
0	1	1
0	0	0

短路操作

逻辑与 双目 L&&R

L	R	N
1	1	1
1	0	0
0	1	0
0	0	0

跳过

逻辑或 双目 L||R

L	R	N
1	1	1
1	0	1
0	1	1
0	0	0

当逻辑与的左操作数为假时，
可以不用执行右操作数了

当逻辑或的左操作数为真时，
可以不用执行右操作数了

总结 当表达式中存在&&或者
||，表达式应该先执行左边
的，看是否触发短路，再考虑
执行右边

```
int i = 1;
int j = 1;
//i == j || (j = 2);
i == j && printf("hello\n");
printf("j = %d\n", j);
return 0;
```

判断某一年是不是闰年

闰年的规则：

- 1、年份能被4整除并且不能被100整除 ---> 闰年
- 2、年份能被400整除 --> 闰年

$\text{year} \% 4 == 0 \ \&\& \ \text{year} \% 100 \neq 0 \ || \ \text{year} \% 400 == 0$

逻辑非 高于 逻辑与 高于 逻辑或

优先级

单目运算符 高于 算术 高于 关系 高于 逻辑

$\ast$ / %	< <= > >=	&&
+ -	== !=	

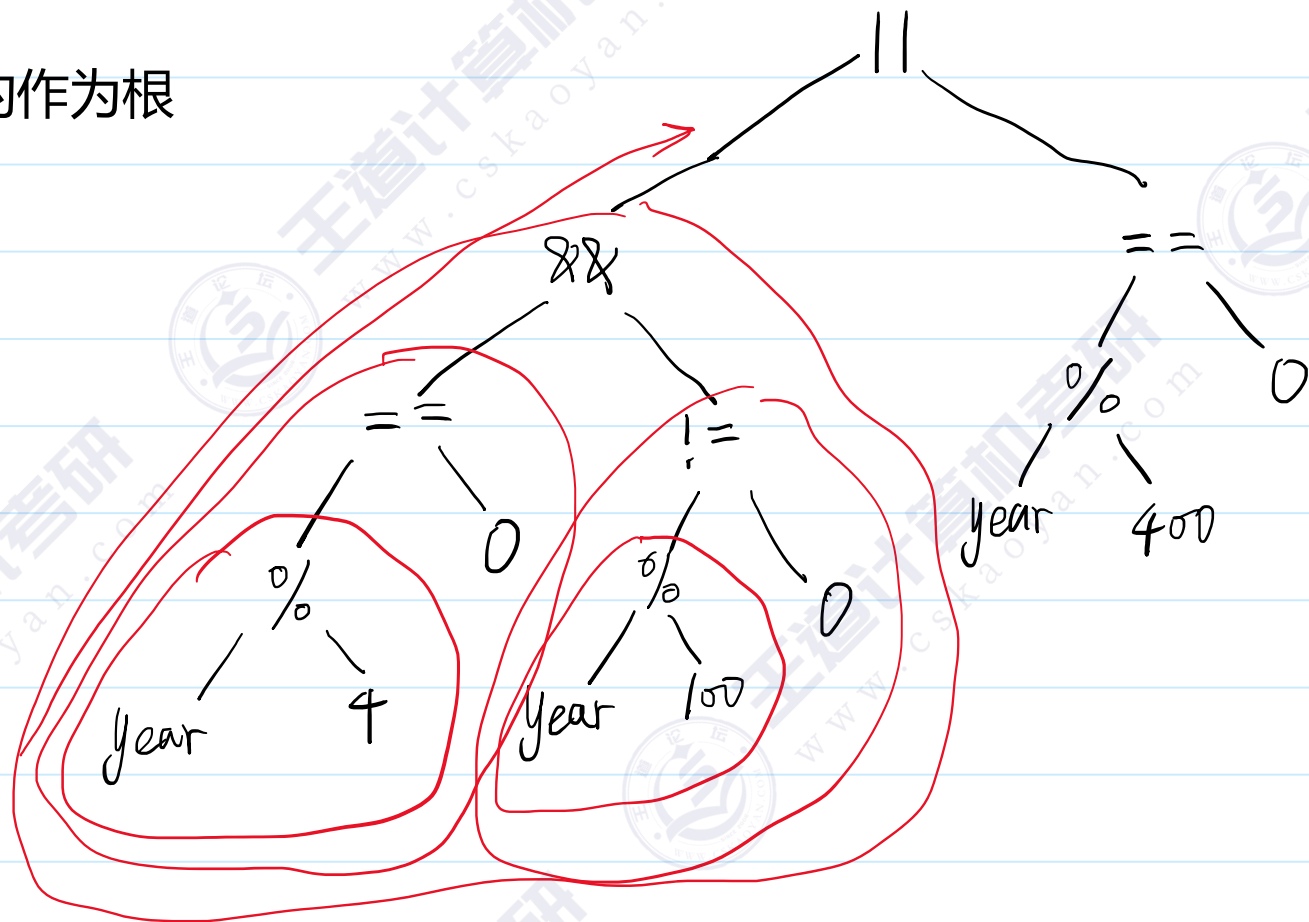
```
year % 4 == 0 && year % 100 != 0 || year % 400 == 0
```

最高的是取余 --> 判断相等和不等 ---> && --> ||

补充的方法 表达式树

$\text{year \% 4 == 0 \&\& year \% 100 != 0 || year \% 400 == 0}$

树 优先级最低的作为根



赋值运算符

$L = R$

优先级比较低

$L \leftarrow R$

把 R 的值 放入 L 的内存空间 里面。

→ 可以是临时数据。

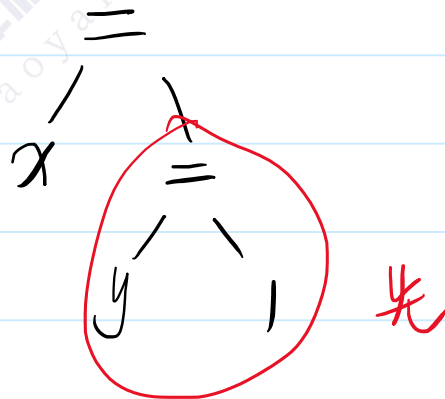
左值。 { ① L 必须代表一片内存空间。
② 该空间可以被修改。

```
int i = 1; // 这个等于号不是赋值，它是初始化
//i = 3 + 4; // 合理
//3 + 4 = i; // 不合理
//i = i + 1; // 合理 i 出现左边表示访问其地址 i 出现在右边表示获取值
//i += 1; // 等价于 i = i + 1;
++i; // 等价于 i = i + 1;
```

赋值运算符的结合性

```
int x,y;  
x = y = 1;
```

赋值运算符的结合性 从右往左



条件运算符

3目 A?B:C

检查A的真假 若为真返回B 若为假就会返回C

```
int a = 10, b = 15;  
int max = a > b ? a : b;  
printf("max = %d\n", max);  
return 0;
```

如果自己写代码，一般用if else

比较三个数的大小？

```
int a = 10, b = 11, c = 1;  
int max = (a > b ? a : b) > c ? (a > b ? a : b) : c;  
// 先a和b做比较，胜者再和c作比较  
printf("max = %d\n", max);
```

逗号运算符

优先级最低的

A, B --> 先执行A再执行B最后返回B

和 A;B; 很像

场景：变量i，要求先改变i的值，再判断i是否合适

```
if( i=i+1 , i == 3){  
}
```

printf("%d",123); ---> 函数参数里面的逗号不是逗号运算符，C语言没有规定参数的处理顺序

自增自减运算符

++ 单目 $\rightarrow$ 操作数必须是左值.

int i=3;

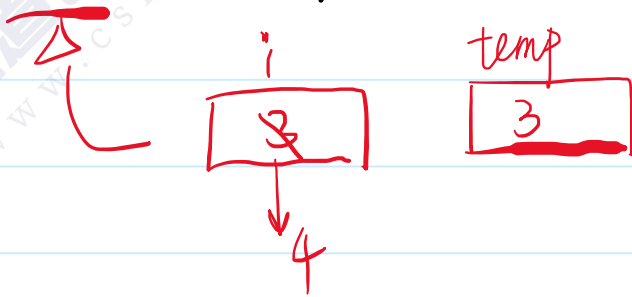
--

	内存内容	返回结果	
++i 前缀自增	4	4	$\Rightarrow$ 先自增后返回
i++ 后缀自增	4	3	$\Rightarrow$ 先返回, 再自增.

```
int i = 3;
int j;
//j = ++i; //前缀自增 先++后返回
j = i++; //后缀自增 先返回后++
printf("i = %d, j = %d\n", i, j);
```

j = i++ * i++; //不好

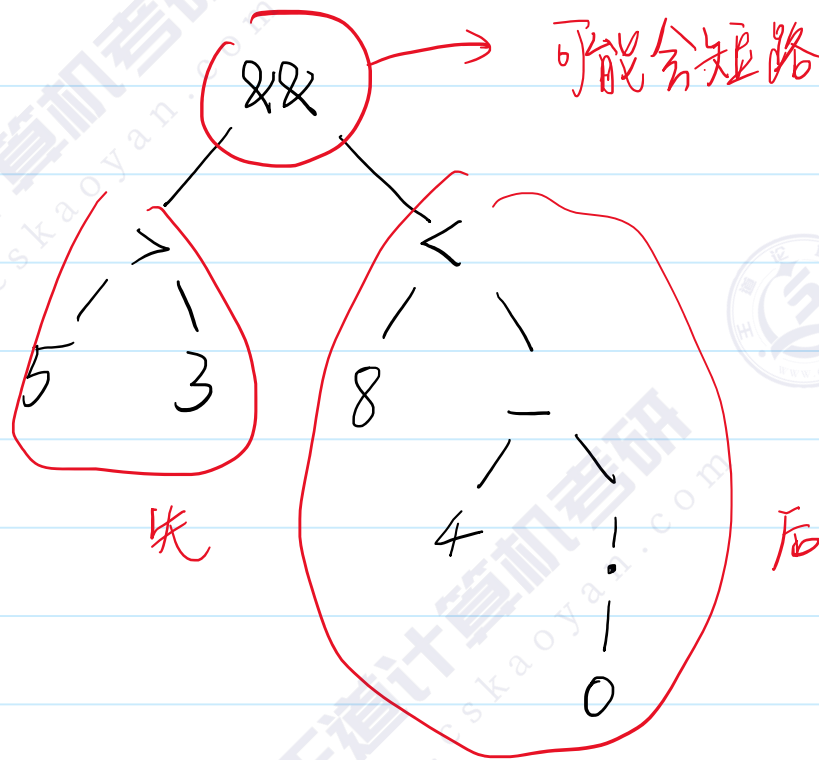
++i 和 i++ 谁更好?



练习题

$5 > 3 \ \&\& \ 8 < 4 - !0$

→ 表达式树



先算 $5 > 3 \rightarrow 1$

$!0 \rightarrow 1$

$4 - 1 \rightarrow 3$

$8 < 3 \rightarrow 0$

最后 $1 \&\& 0 \rightarrow 0$

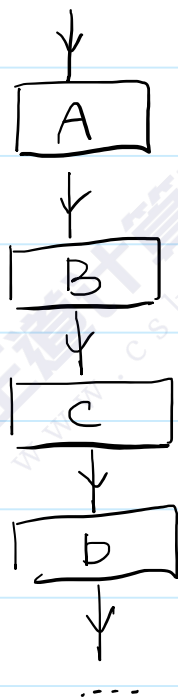
- 1、需要按照优先级整理表达式树
- 2、几个特殊运算符 $\&\& \ ||$ ，规定了执行顺序

最后 1&&0 -->0

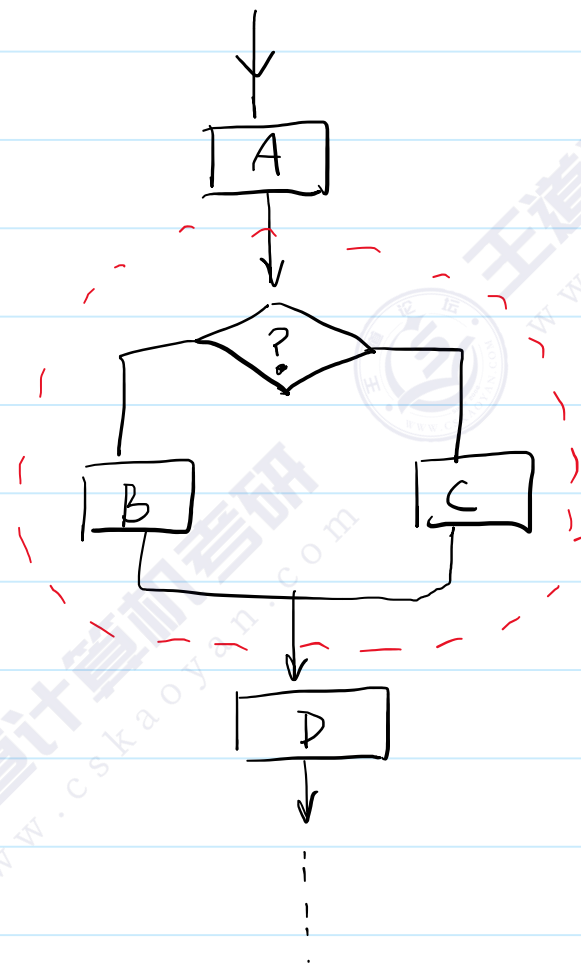
2、几个特殊运算符 && || ，规定了执行顺序

选择结构

....
A;
B;
C;
D;
....



顺序结构



选择结构

if

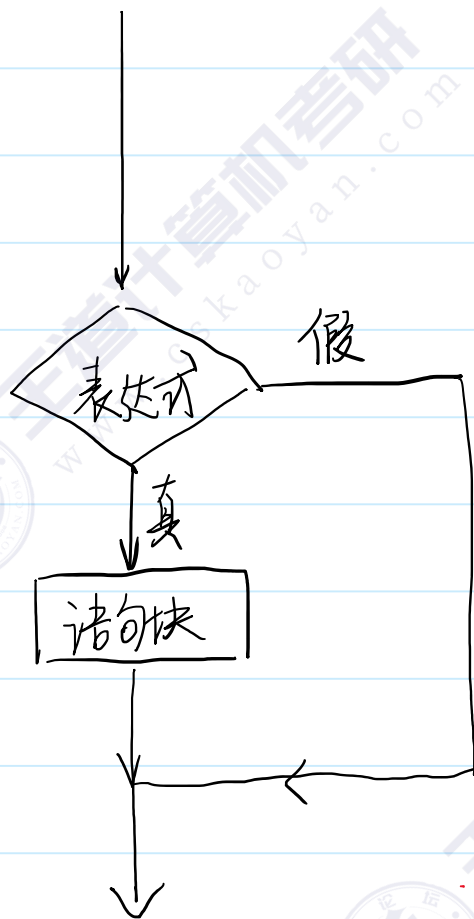
switch

单分支if

if(表达式)

语句块

- ① 一个语句
② { 多语句 }



```
#define _CRT_SECURE_NO_WARNINGS
```

```
#include <stdio.h>
```

```
int main() {
```

```
    int i;
```

```
    scanf("%d", &i);
```

```
    if (i == 1) {
```

```
        printf("i is 1!\n");
```

```
        printf("i is 1!\n");
```

```
    } //编程要求, 我们自己写的代码, 不能省略花括号
```

```
    return 0;
```

```
}
```

苹果公司的bug

```
10     if ((err = SSLHashSHA1.update(&hashCtx, &signedParams)) != 0)
11         goto fail;
12     goto fail;
13     if ((err = SSLHashSHA1.final(&hashCtx, &hashOut)) != 0)
```

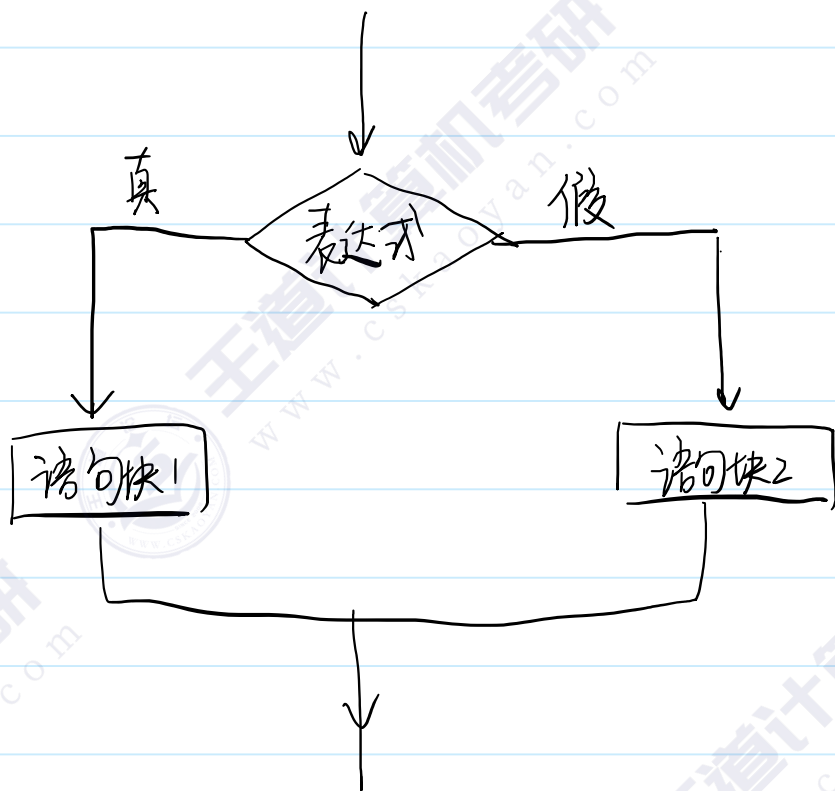
两个分支的if

if(表达式)

语句块1

else

语句块2



```
int a = 10, b = 15;  
if (a > b) {  
    printf("max = a , a is %d\n", a);  
}  
else {  
    printf("max = b , b is %d\n", b);  
}
```

比较三个数的例子

```
int a = 30, b = 25, c = 20;
if (a > b) {
    if (a > c) {
        printf("a is max. a = %d\n", a);
    }
    else {
        printf("c is max. c = %d\n", c);
    }
}
else {
    if (b > c) {
        printf("b is max. b = %d\n", b);
    }
    else {
        printf("c is max. c = %d\n", c);
    }
}
```

更多分支的if

只需要在else的语句里面嵌套使用if，就可以实现更多分支的if了

```
int height;
scanf("%d", &height);
if (height < 160) {
    printf("He is short!\n");
}
else if (height <= 185) {
    // 这个分支要满足 >= 160 && <=185
    printf("He is middle-sized!\n");
}
//else if () {
//}
else {
    // > 185
    printf("He is tall!\n");
}
return 0;
```

不加花括号可能会出现的问题

```
int i = 0;  
if (i > 1)  
    if (i < 10)  
        printf("i is between 1 and 10\n");  
else  
    printf("i is less than 1\n");
```

就近匹配

看上去，if和else匹配 x 实际上并不匹配

```
int i = 0;  
if (i > 1) {  
    if (i < 10) {  
        printf("i is between 1 and 10\n");  
    }  
}  
else {  
    printf("i is less than 1\n");  
}
```

Microsoft Visual Studio 调试控制台

i is less than 1

记得加花括号

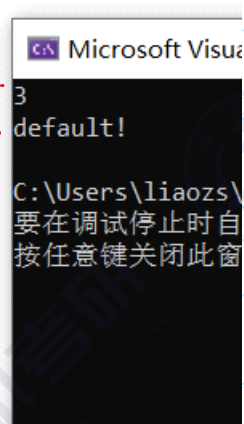
switch

固定的多分支

```
switch(表达式){  
case 值1:  
    ...  
case 值2:  
    ...  
...  
default:  
    ...  
}
```

跳转

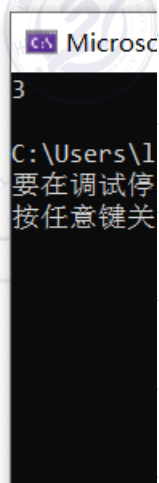
```
int i = 3;  
switch (i) {  
case 1:  
    printf("1\n");  
case 2:  
    printf("2\n");  
case 3:  
    printf("3\n");  
default:  
    printf("default!\n");  
}  
return 0;
```



switch当中的break

switch语句块的代码遇到break会立刻跳出语句块

```
int i = 3;
switch (i) {
case 1:
    printf("1\n");
    break;
case 2:
    printf("2\n");
    break;
case 3:
    printf("3\n");
    break;
default:
    printf("default!\n");
    break;
}
```



场景题

给出年份和月份，求这个月有多少天？

1 3 5 7 8 10 12 ==> 31天

4 6 9 11 ==> 30天

2 ==> 28或者是29天

```
int year, mon;
scanf("%d%d", &year, &mon);
int dom;
switch (mon) {
    case 1: case 3: case 5:
    case 7: case 8: case 10:
    case 12:
        dom = 31;
        printf("dom = %d\n", dom);
        break;
    case 4: case 6: case 9: case 11:
        dom = 30;
        printf("dom = %d\n", dom);
        break;
    case 2:
        dom = 28 + (year % 400 == 0 || year % 4 == 0 && year % 100 != 0);
        printf("dom = %d\n", dom);
        break;
    default:
        printf("error!\n");
}
```

goto

实现函数内部的跳转 --> 重复做一些事情 (循环结构)

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
int main() {
    int i = 1;
    int total = 0;
label:
    total += i;
    ++i;
    if (i <= 100) {
        goto label;
    }
    printf("total = %d\n", total);
}
total = 5050
```

先写标签，再写goto，可以实现循环

goto语句应该要放在if结构里面，否则会产生死循环（死循环需要用任务管理器排除）

goto 实现循环的问题

迪杰斯特拉 goto有害

- 1、代码的可读性下降
- 2、性能问题（破坏了局部性）

我们一般情况不使用goto，就算是用goto，也不会用它来实现循环

goto一般的场景是做快速跳转——可以用goto离开多重循环，我们希望只用goto做向后跳转。

while循环

while(表达式)

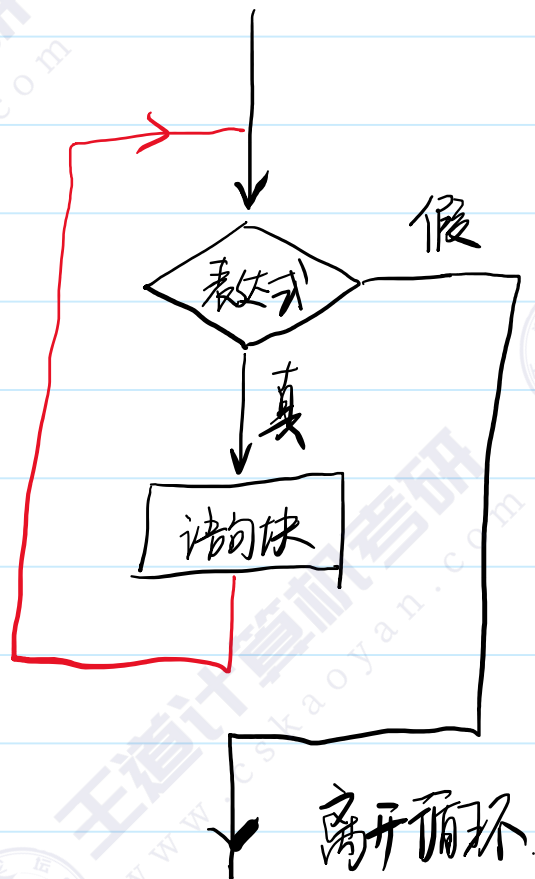
语句块



循环体

入口条件

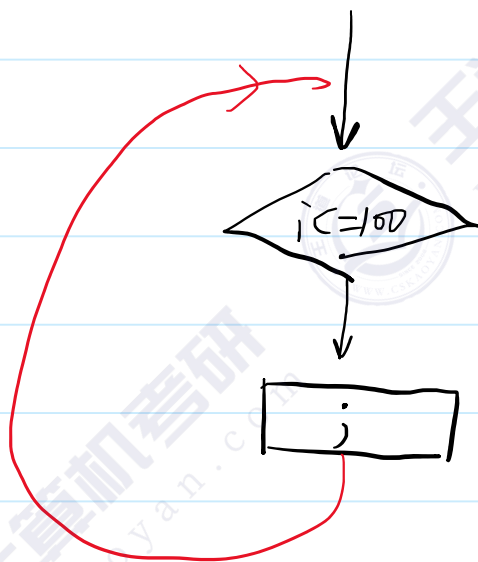
- ① 单个句子
- ② { 多个句子 }



为什么它会死循环?

```
while (i <= 100); { //先写循环体, 再写入口条件  
    total += i;  
    ++i;  
}
```

离开循环.



如果不需要一个死循环, 那么我们需要让循环体去改变入口条件, 让它趋向于假.

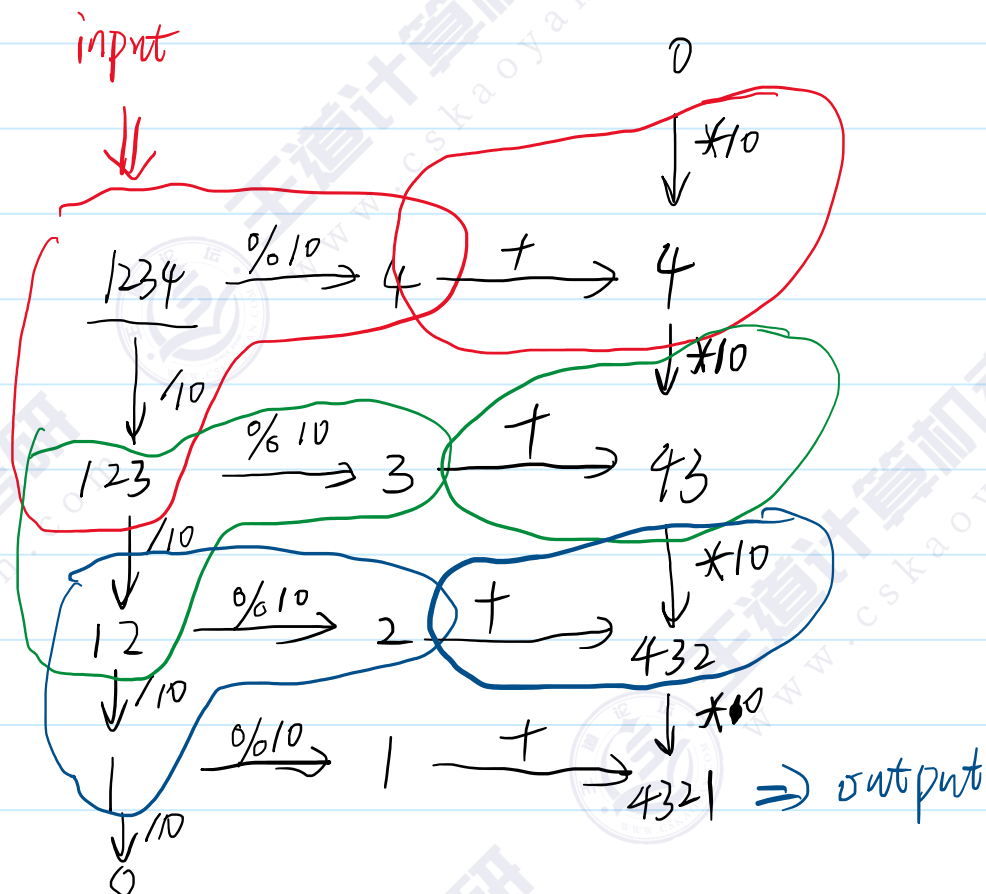
题目 翻转整数

输入一个整数，然后输出翻转之后的结果

1234 --> 4321

12345 --> 54321

先写循环体



使用循环解决问题的思路

- 1、代码中肯定存在重复的行为
- 2、先想循环体，把问题的中间状态列一个表格，观察各个数据的变化，设计合理的变量
- 3、入口条件，重点关注最后一次循环体（边界问题）

代码

```
//翻转整数
int num;
scanf("%d", &num);
int input = num;
int output = 0;
while (input > 0) {
    output *= 10;
    output += input % 10;
    input /= 10;
}
printf("output = %d\n", output);
```


题目 小写转大写

输入一行字符串，将其中的小写字母转成大写字母

hello world --> HELLO WORLD

NiHao123 --> NIHA0123

边界问题?

N --> N

i --> I

H --> H

a --> A

o --> O

1 --> 1

2 --> 2

3 --> 3

NiHao123换行

→ 读到换行停止.

scanf("%c", ...)

```
while (scanf("%c", &ch), ch != '\n') {  
    |  
}
```

一个表达式. 执行多个语句. => 逗号运算符.

入口条件做多件事情可以依靠逗号运算符

//读取一行

char ch;

while (scanf("%c", &ch), ch != '\n') {

if (ch >= 'a' && ch <= 'z') {

ch -= 32; //小写字母 转成 大写字母

}

printf("%c", ch);

}

printf("\n");

return 0;

Microsoft Visual Studio 调试控制台

NiHao123

NIHA0123

for循环

执行100次的循环

```
① int i = 0; ②  
while(i < 100){  
    // 循环体  
    ++i; ③  
}
```

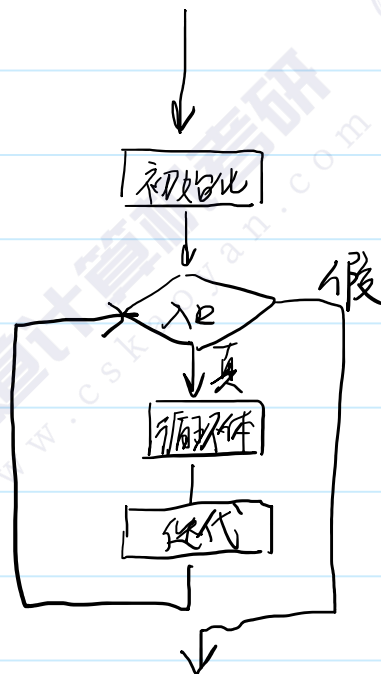
```
int total = 0;  
for (int i = 1; i <= 100; ++i) {  
    total += i;  
}  
printf("total = %d\n", total);
```

Microsoft Visual Studio 调试控制台

total = 5050

for希望将循环的代码和主业务的代码分离
更加清晰，方便阅读

for(初始化;入口条件;循环变量迭代)
语句块



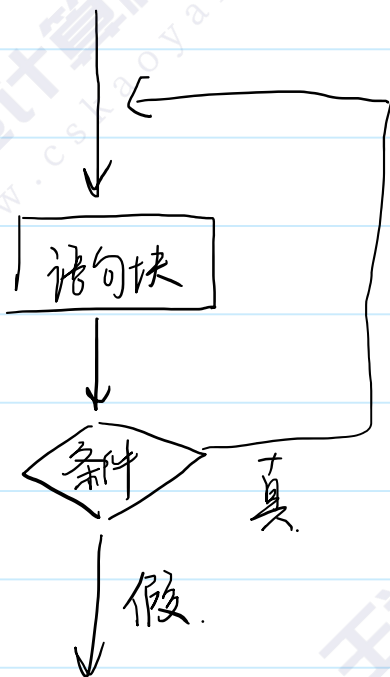
do while循环

do

语句块

while(条件);

先执行语句块，再检查条件



```
int total = 0;
int i = 1;
do {
    total += i;
    ++i;
} while (i <= 100);
printf("total = %d\n", total);
```

```
do {
    printf("Hello\n");
} while (0);
```

↑
执行一次

continue关键字

continue用在循环体里面

程序运行的时候遇到continue，就会跳过本次循环体，
直接开始下一次循环体

场景：从2，4，6，8 ... 100

```
int i = 1;
int total = 0;
while (i <= 100) {
    if (i % 2 != 0) {
        ++i;
        continue; //continue之前记得迭代条件
    }
    total += i;
    ++i;
}
printf("total = %d\n", total);
```

```
int total = 0;
for (int i = 1; i <= 100; ++i) {
    if (i % 2 != 0) {
        continue;
    }
    total += i;
}
printf("total = %d\n", total);
```

break关键字

break之前在switch里面用过

现在是一种新用法

break只能出现循环体里面

当程序运行到break的时候，程序会跳出单层循环

举例子 $1+2+3+4+5+\dots+n > 200$

不确定次数的循环

```
int total = 0;
int i = 1;
while (1) {
    // 死循环 + 循环体里面break ==> 不确定次数的循环
    total += i;
    if (total > 200) {
        break;
    }
    ++i;
}
printf("total = %d, i = %d\n", total, i);
return 0;
```

Microsoft Visual Studio 调试控制台

total = 210, i = 20

枚举思想

面对一个问题的时候，直接将问题的所有可能性全部列出来，一个一个地去检查是否符合题目的要求 ---> 暴力

循环

水仙花数

Description

“水仙花数”是指一个三位数，其各位数字的立方和等于该数本身。例如，153是一个水仙花数，因为 $1^3 + 5^3 + 3^3 = 153$ 。

请编写程序，找出所有的三位水仙花数。

100 ~ 999

a 百

b 十

c 个

$$\overline{abc} = 100 \times a + 10 \times b + c \quad ? = a^3 + b^3 + c^3$$

a 1~9
b 0~9
c 0~9 } 三重循环

找完数

Description

完数是指除本身以外的因子之和等于其本身的数。

任给一个自然数 n ,求 n 以内的所有完数。如果找不到,则输出"No"

1~n

Input

一行: 一个整数 n . ($0 < n < 10000$)

Output

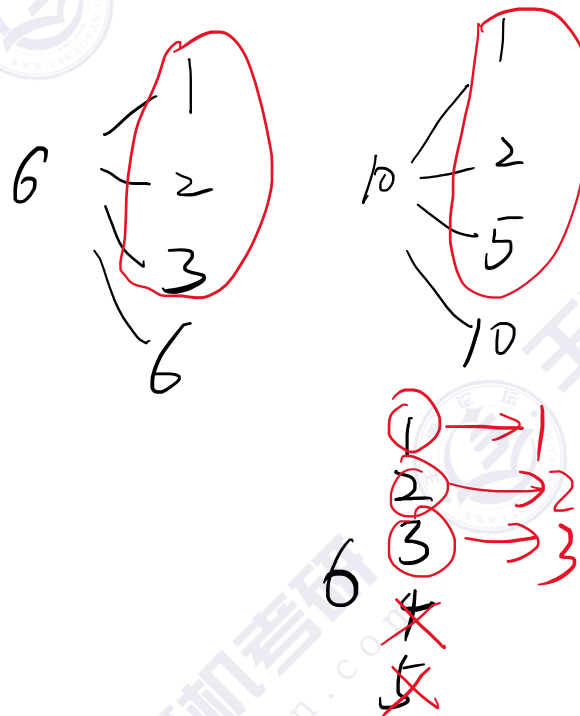
有若干行: 每一行一个完数。如果没有完数,则输出No

Sample Input 1

10

Sample Output 1

6



检查一个数是否为质数

Description

输入一个正整数，检查该数是否为质数

Input

输入一个正整数

Output

输出Y或者N

Sample Input 1

34

Sample Output 1

N

能不能优化

从 $2 \sim n-1$ ----> 有没有办法少检查几次

假如一个数不是质数 $x = a * b$ (a, b 不是 1 也不是 x)

不可能 $a > \sqrt{x}$ 并且 $b > \sqrt{x} \implies$ 假如一个数 x 不是质数, 那么它至少有一个约数是小于等于 $\sqrt{x}$